

Exact Unknot Recognition with Budgeted Bracket Filters and a Compiled $\mathbb{F}_2$ Cobordism Kernel

An implemented and tested acceleration of `Knots.zip/fast/`

Prepared for Vladimir Reshetnikov
Computational research and implementation: OpenAI

18 September 2026 — Software version 0.2.0

Abstract

We supply a working replacement for the archive’s exact unknot recognizer, not a quasi-polynomial claim or a hierarchy-algorithm stub. The main practical change is to avoid constructing Khovanov homology when an inexpensive, exact invariant obstruction already proves knottedness. A sparse modular determinant and a budgeted Kauffman-bracket scan are followed, when appropriate, by an exact integer bracket evaluation; inconclusive tests always fall through to the complete backend. Inside that backend, topology compilation, characteristic-two algebra, bounded memoization, and an equivalent heap-based scan planner reduce work without changing the computed ranks. The planner improves from $O(rn^2)$ to $O(rn \log n)$ for r starting points, and the descending test becomes linear. We also correct the writhe convention for general valid PD inputs. In 138 paired cold-process trials, the median full-recognition speedup is 568.54 on a 22-crossing connected sum of two Conway knots; raw Khovanov computation is 42.85 times faster on a 1024-crossing unknot chain. Regressions and unresolved stress cases are reported. Forty-two tests pass, including independent state sums and reduced-cube calculations. The general worst-case bound remains exponential. We identify precisely why small scan width does not by itself make the explicit Khovanov backend quasi-polynomial, and separate the implemented results from the additional obligations in Lackenby’s hierarchy approach.

Contents

1	Scope, outcome, and reproducibility	3
1.1	The retained input contract	3
1.2	What is new, and what is inherited	3
2	Lackenby’s material and the quasi-polynomial target	4
3	The accelerated recognition pipeline	4
4	A sparse modular determinant obstruction	5
5	Bracket filtering by frontier aggregation	6
5.1	Conventions and the necessary identity	6
5.2	The state and transition	6
5.3	Exact integer mode without rational denominators	7
5.4	Width-sensitive and budget-sensitive complexity	8
6	A necessary orientation repair	8
7	Faster exact-equivalent preprocessing and planning	9
7.1	A linear descending test	9
7.2	Heap-based scan ordering	9
8	Compiling the characteristic-two cobordism algebra	10
8.1	Representation and cancellation	10
8.2	Every unit is its own inverse	10
8.3	Separating topology from dot data	11
8.4	Why the final rank detects the unknot	11
9	Why frontier width alone cannot bound the explicit backend	12
10	Paired measurements, including regressions	13
10.1	Protocol	13
10.2	What the results establish	15
10.3	Counters that should not be compared naively	15
11	Validation and the limits of that validation	15
12	Using the package and interpreting evidence	17
12.1	Commands	17
12.2	Limits and ablation switches	17
12.3	What the verifier certifies	18
13	Conclusions and remaining engineering opportunities	18

1 Scope, outcome, and reproducibility

The target is the Python package in `fast/` of the supplied `Knots.zip` [1]. The original seven module files are preserved byte-for-byte in `reference/baseline_fastunknot/`; they are imported under a different package name solely to allow paired experiments. The modified package is `fastunknot/`. It has no runtime dependency outside Python’s standard library and is designed for Python 3.10 or later. This delivery was executed with Python 3.13.5.

The outcome has three distinct levels. First, the complete recognizer is substantially faster on several difficult-for-Alexander inputs. Second, there are proved improvements to individual algorithms and a proved width-sensitive bound for the new one-sided bracket filter. Third, *there is no general $n^{O(\log n)}$ bound for the delivered recognizer*. The last point is not an assumption about what future research can achieve: it describes the actual implementation and the bounds established here.

The most important design constraint is that UNKNOWN, equality of an invariant with the unknot’s value, and exhaustion of a filter budget must never be confused with a topological verdict. No floating-point computation is used for a mathematical decision. The algorithm does not consult a knot table or recognize benchmark fixtures by name.

1.1 The retained input contract

A crossing is a counterclockwise tuple (a, b, c, d) , with under-strand $a-c$ and over-strand $b-d$. Each edge label occurs twice. The validator normalizes labels, checks that following opposite ports gives a single component, and checks that the rotation system is spherical. For a connected four-valent n -crossing map, the latter check is the Euler condition $F = n + 2$. The input `{"pd": []}` represents one crossing-free circle, not an empty link. Links and virtual diagrams are rejected. The inherited braid and grid input formats are retained. Complexity bounds below use normalized edge labels; reading a noncanonical encoding additionally costs its input bit length. Grid rows run bottom to top and vertical segments pass over horizontal segments.

```
{"pd": [[1,4,2,5], [3,6,4,1], [5,2,6,3]]}
{"braid": {"strands": 3, "word": [1,-2,1,-2]}}
{"rows": [[0,2], [1,3], [2,4], [0,3], [1,4]]}
```

1.2 What is new, and what is inherited

The inherited recognizer validates, performs crossing-decreasing Reidemeister I/II moves, tests for a descending presentation, computes the full Alexander polynomial, and finally computes reduced Khovanov rank through an unreduced Bar–Natan scan. We retain the underlying topology and elimination strategy. The new work consists of executable filters, algebraic specialization and caching, faster exact-equivalent planning, a linear descending test, convention repairs, additional validation, a result verifier, and reproducible experiments. The Kauffman bracket, Bar–Natan’s local method, and Khovanov unknot detection are established mathematics, not discoveries claimed by this report [5, 4, 6].

2 Lackenby’s material and the quasi-polynomial target

The attached talk is dated February 2021, whereas the attached preprint is *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, dated 25 July 2026. These are different documents and should not be conflated. The talk announces the quasi-polynomial result and describes an accelerated construction involving hierarchical multi-surfaces, Cheeger regions, cutting, weak reductions, and simplification. The preprint gives a hierarchy-based algorithm and further structural results. In Section 9 it explicitly separates an iteration count from a running-time estimate: its initial count uses a maximal hierarchy length L and maximal pattern complexity g , with bound $L(g+1)^L$ [2, 3].

For clarity, the following is an elementary conditional calculation, not a new proof of the announced topological algorithm.

Proposition 2.1 (A sufficient complexity contract). *Suppose a constructive hierarchy algorithm on an n -crossing input has at most $L(g+1)^L$ iterations, where $L \leq C \log(n+2)$ and $g \leq (n+2)^C$. Suppose all represented data and every iteration can be computed using at most $n^{O(\log n)}$ bit operations and comparable space. Then its running time is $n^{O(\log n)}$.*

Proof. The logarithm of the iteration bound is $\log L + L \log(g+1) = O((\log(n+2))^2)$. Multiplying it by another $\exp(O((\log(n+2))^2))$ bound leaves the same asymptotic class. $\square$

A correct implementation of that route must supply the represented handle structures and boundary patterns, exact surface operations, a decreasing potential, size bounds, and the accelerated depth bound together. A finite lexicographically decreasing counter is not sufficient: its length, digit sizes, coordinate bit lengths, and the work required to find each next move matter. Section 10 and the later sections of the preprint develop refinements; our implementation does not purport to instantiate them or to turn the talk’s flowchart into executable primitives.

The delivered package instead makes genuine progress on the requested fallback goal: better practical performance and better asymptotic subroutines for the provided implementation. There are no unimplemented hierarchy functions hidden behind the command-line interface. Section 9 also exhibits a concrete obstruction to a tempting but invalid alternative proof based only on the number of frontier matchings.

3 The accelerated recognition pipeline

Write n for the number of crossings after optional Reidemeister reduction when discussing filter budgets. The actual pipeline is:

1. Validate the original diagram; optionally simplify by legal R1/R2 moves and run the descending test. These stages may prove UNKNOT.
2. Evaluate an Alexander minor at $t = -1$ modulo $2^{31} - 1$ using sparse Gaussian elimination. A value other than ± 1 proves KNOTTED.
3. Compute a budgeted bracket value at $A = 2$ modulo $2^{31} - 1$. A mismatch with the unknot value proves KNOTTED.
4. If that modular evaluation completed but matched, try an exact integer scaled bracket evaluation at $A = 2$, again with a budget. A mismatch proves KNOTTED.

5. Compute the original full exact Alexander polynomial, unless disabled. A nonunit normalized polynomial proves **KNOTTED**.
6. Run the improved complete Khovanov backend. Reduced rank one proves **UNKNOT**; any other rank proves **KNOTTED**.

Each bracket evaluation receives a default transition budget

$$B(n) = 10000 + 200n \tag{1}$$

and a limit of 4096 live pairing states. A transition means one smoothing of one current pairing. Reaching either ceiling discards the incomplete test and continues the pipeline. It does not consume the right to try the complete backend. The second, exact evaluation is skipped when the first did not complete, to avoid predictably repeating an already oversized computation.

The determinant precedes the bracket because it often rejects nontrivial knots at very low cost. The bracket precedes the full polynomial because it handles the Alexander-trivial fixtures and their connected sums particularly well. This order is a practical choice, not a theorem of optimality. It can be slower on small inputs whose Alexander polynomial already rejects them; the measurements below include these cases.

Theorem 3.1 (Soundness and completeness of the wrapper). *On a validated classical one-component input, assuming the stated exact algebraic operations and the retained topological algorithms, every returned **UNKNOT** or **KNOTTED** is correct. Without an overall time or Khovanov-object ceiling, the recognizer is complete. The default finite budgets for the new filters do not affect completeness.*

Proof. R1/R2 moves preserve knot type. A descending closed diagram represents the unknot: cut at its descending basepoint, place the successive traversed arc at strictly decreasing height, and then straighten the resulting unentangled arc before closing it. Sections 4 and 5 prove that each new rejection is a necessary-identity violation for the unknot. A completed full Alexander calculation is an established invariant obstruction. Every inconclusive filter leaves the same reduced diagram for the next stage. The retained Khovanov construction is finite, and the rank-one detection criterion is justified in Section 8.4. With imposed resource ceilings an unfinished complete calculation returns **UNKNOWN**, not a verdict. $\square$

4 A sparse modular determinant obstruction

Collapse the over-strand edges at each crossing to obtain the Wirtinger arcs. For a nonempty one-component n -crossing diagram there are n such arcs. An Alexander matrix has one row per crossing; up to the crossing convention its three contributions are $1 - t$ on the over-arc and $t, -1$ on the two under-arcs. At $t = -1$, the two under entries agree, so crossing signs and their incoming order disappear. The row becomes

$$2e_{\text{over}} - e_{\text{under},1} - e_{\text{under},2}. \tag{2}$$

Contributions are added when two arc indices coincide. Delete one row and one column. The resulting minor is $\Delta_K(-1)$ up to sign, because the usual unit ambiguity $\pm t^j$ becomes ± 1 .

Proposition 4.1. *Let $p = 2147483647$. If the determinant of this minor modulo p lies outside $\{1, p-1\}$, the knot is nontrivial.*

Proof. The Alexander polynomial of the unknot is one up to the unit ambiguity. Therefore every indicated minor for an unknot specializes to $+1$ or -1 . Reduction modulo p preserves this necessary identity. Its violation is an exact disproof, not a probabilistic guess. $\square$

Rows are sparse Python dictionaries. Gaussian elimination chooses the first available nonzero entry in each fixed pivot column, swaps rows when necessary, multiplies the running determinant by the pivot, and updates only stored tail entries. Modular inverses are computed exactly. In the dense worst case this uses $O(n^3)$ field operations. The recognizer uses only the fixed prime above; the low-level determinant routine assumes its optional modulus is prime and is not a general composite-modulus determinant routine.

A determinant congruent to ± 1 is inconclusive. This includes every Alexander-trivial knot. The full Alexander polynomial is retained because $\Delta_K \neq 1$ can hold even when the determinant test does not reject.

5 Bracket filtering by frontier aggregation

5.1 Conventions and the necessary identity

For A invertible, put

$$\delta = -A^2 - A^{-2}. \quad (3)$$

We use the unnormalized bracket $\mathcal{B}_D(A)$: every closed state circle contributes δ , including the first. At a PD crossing (a, b, c, d) , the zero smoothing pairs $(a, b), (c, d)$ with weight A , and the one smoothing pairs $(a, d), (b, c)$ with weight A^{-1} . Thus, for $n > 0$,

$$\mathcal{B}_D(A) = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)}, \quad (4)$$

where $c(s)$ is the number of circles of the completely smoothed diagram. The zero-crossing input has bracket δ .

With the oriented writhe $w(D)$ in the PD convention implemented here, an unknot diagram must satisfy

$$\mathcal{B}_D(A) = \delta(-A^3)^{w(D)}. \quad (5)$$

The power is computed in the ring in question; a negative writhe is handled by inverting the unit $-A^3$. In particular, the modular test does not divide by δ , which might be zero.

For completeness, the local algebra underlying this identity can be checked without a knot table. Let e_i be the Temperley–Lieb smoothing, satisfying $e_i^2 = \delta e_i$ and $e_i e_{i+1} e_i = e_i$. A crossing and its inverse have operators $AI + A^{-1}e_i$ and $A^{-1}I + Ae_i$; their product is $I + (A^2 + A^{-2} + \delta)e_i = I$, proving R2 invariance. Expanding the adjacent three-crossing products, their difference is a scalar multiple of $A^2 + \delta + A^{-2} = 0$, proving R3 invariance. The two R1 curls evaluate to $-A^3$ and $-A^{-3}$. Multiplication by $(-A^3)^{-w(D)}$ removes these factors. The crossing-free circle then proves (5). This is the standard bracket construction [5].

5.2 The state and transition

After some crossings have been processed, keep the edges incident to exactly one processed crossing as the frontier. A partial smoothing consists of closed circles and paths joining frontier endpoints.

Closed circles are immediately evaluated. The remaining information is a perfect matching m of the frontier and a coefficient $v(m)$ in the chosen ring.

To add a crossing, take the union of the current matching and one smoothing's two local arcs. This is a degree-at-most-two multigraph: its path components give a new matching m' , and its cycle components give a number c of newly closed circles. Add

$$v(m)A^{1-2s}\delta^c \quad \text{to the coefficient of } m', \quad s \in \{0, 1\}. \quad (6)$$

An edge may occur twice in a PD row. Degrees and edges are counted with multiplicity, so a self-loop smoothing is correctly recognized as a circle. The function `attach` implements this transition independently of the Khovanov backend's gluing routine.

Lemma 5.1 (Exact aggregation). *After every processed prefix, the coefficient stored at m is the sum of the weights of all partial smoothing choices with residual matching m , with closed circles already evaluated. Consequently the completed scan equals (4) for every crossing order.*

Proof. Initially there is one empty partial choice with weight one. Given the invariant for a prefix, each choice extends uniquely in two ways at the next crossing. Equation (6) contributes exactly the new crossing weight and the newly completed circles. Future attachments depend only on the residual pairing, not on how it arose, so equal pairings may be summed. At the end all components are closed and the only possible matching is empty. The zero-crossing circle is treated separately. $\square$

In modular mode, coefficients are reduced modulo p after every addition. Terms whose aggregate coefficient vanishes are omitted. This omission is safe by linearity. A completed mismatch in (5) returns a knottedness witness. A matching value proves nothing about triviality.

5.3 Exact integer mode without rational denominators

A fixed modular value can collide with the unknot value. To avoid relying only on finite-field information, the implementation also has an exact evaluation at any integer $a \geq 2$, normally $a = 2$. Put $D = -(a^4 + 1)$, so $\delta = D/a^2$. Instead of storing rational coefficients after j crossings, store them multiplied by a^{5j} . The transition weight becomes

$$a^5 a^{1-2s} (D/a^2)^c = a^{6-2s-2c} D^c. \quad (7)$$

At most two circles close when the two smoothing arcs are attached; hence $c \leq 2$, and the exponent in (7) is nonnegative. All intermediate arithmetic is integral.

For $n \geq 1$ the expected scaled unknot value is

$$U_D = (-1)^{w(D)} D a^{5n+3w(D)-2}. \quad (8)$$

Since $|w(D)| \leq n$, its exponent is at least $2n - 2$. For $n = 0$ the scale is a^2 and both values equal D . Completed values are serialized as hexadecimal strings, avoiding decimal digit-conversion limits on very large Python integers. These are exact integer comparisons, not numerical approximations.

For fixed a , each transition multiplier has bounded absolute size, and at most 2^n terms contribute. Therefore every coefficient has $O(n)$ bits. For variable a , a bound is $O(n(1 + \log a))$ bits. An equality at one exact point is still only inconclusive: the algorithm does not assume that the Jones polynomial, or any one of its evaluations, detects the unknot.

5.4 Width-sensitive and budget-sensitive complexity

Let w be the largest frontier size of the supplied order, and define

$$M(w) = (w-1)!! = \frac{w!}{2^{w/2}(w/2)!} \quad (w \text{ even}), \quad M(0) = 1. \quad (9)$$

There are at most $M(w)$ perfect matchings at a step. We use this safe bound, not a Catalan bound that would require an additional disk-boundary embedding hypothesis. An arbitrary processed region need not be a disk.

Proposition 5.2 (Complexity of the bracket filter). *For fixed modulus and evaluation point, an uncapped scan of width w uses $O(nM(w) \text{poly}(w))$ word operations, apart from order construction. It stores $O(M(w) \text{poly}(w))$ words. Since $M(w) = \exp(O(w \log(w+1)))$, this is a width-sensitive bound for the filter. At a fixed integer point, a conservative bit bound is $\text{poly}(n, w)M(w)$. With the default transition budget and state ceiling, the added filter work is polynomial in n even for unbounded width.*

Proof. Each state has two transitions per crossing. The degree-two connectivity calculation, canonical sorting, and hashing have polynomial cost in w ; $O(w^2)$ is a sufficient conservative bound for the simple implementation on word-sized labels. Modular scalar operations have fixed-size operands. The integer case additionally uses the linear coefficient bit bound above. Under the default budget there are only $O(n)$ transitions in each of at most two evaluations, and $w \leq 2n$. The matching keys and the exact coefficients therefore have polynomial size. Crossing-order construction is polynomial as well. Interrupting the scan cannot increase its work beyond these caps except for checking and constructing the bounded current transition. $\square$

This theorem is deliberately not stated for the Khovanov complex: its object multiplicities are additional data, as Section 9 shows.

6 A necessary orientation repair

The old `Diagram.signs()` reads which over-port is incoming but implicitly assumes that the incoming under-port is slot zero. That assumption is not part of the accepted PD format. Rotating a row by two positions preserves the crossing, but can invalidate that implicit orientation choice.

For each crossing let $u \in \{0, 2\}$ and $o \in \{1, 3\}$ be the incoming under- and over-port indices along an oriented traversal. The corrected sign is

$$\epsilon = \begin{cases} +1, & o - u \equiv 3 \pmod{4}, \\ -1, & o - u \equiv 1 \pmod{4}. \end{cases} \quad (10)$$

Reversing the knot orientation adds two to both incoming indices, so the sign is unchanged. Rotating a PD row by two has the same invariance. A mirror reverses the sign. This is essential for (5); a writhe error would turn a correct bracket computation into a false rejection.

For example, the three-crossing braid $[1, 1, 1]$ is converted by this package to

$$(5, 4, 0, 1), \quad (1, 0, 2, 3), \quad (3, 2, 4, 5).$$

Rotating the first row by two leaves the new writhe equal to -3 , whereas the old function changes from -3 to $+1$. The signs here reflect the package's PD and braid conversion convention; the example is not an assertion about a different external braid-sign convention.

The original Alexander code always assigns its under-arc variables using PD slots zero and two. Its old sign convention is compensated by that assignment. Accordingly, this report does *not* infer that the old Alexander polynomial or the old recognizer was wrong from the writhe bug alone. The new Alexander matrix uses the genuine oriented sign *and* swaps the incoming/outgoing under variables when the incoming under-port is slot two. Exact comparisons on rotated inputs confirm preservation of the polynomial.

7 Faster exact-equivalent preprocessing and planning

7.1 A linear descending test

Fix one orientation and index its $2n$ crossing visits cyclically. If a crossing is visited over at index o and under at index u , precisely the start indices in the circular interval $(o, u]$ encounter its underpass first. Add one on each such bad interval with a difference array, splitting a wrapped interval into two ordinary intervals. Prefix sums give the number of bad crossings for every basepoint. A zero is a descending presentation. Repeat for the reverse orientation and choose the smallest valid dart, matching the old tie-breaking rule.

There are n intervals and $2n$ starts in each of two orientations. The running time and auxiliary space are $O(n)$, compared with up to $4n$ separate $O(n)$ traversals in the old test. This optimization does not alter the R1/R2 simplifier; that simplifier still recomputes faces and validates diagrams after moves, and remains a polynomial but potentially significant cost.

7.2 Heap-based scan ordering

For an unprocessed crossing i , let s_i count its edges shared with the current processed region, let ℓ_i count self-loop pairs at that crossing, and let b be the current boundary size. The old greedy score is

$$(s_i, -(b + 4 - 2s_i - 2\ell_i), -i).$$

For a fixed step its ordering is exactly the ordering of $(s_i, \ell_i, -i)$. Self-loop counts are fixed; only shared counts change.

Build edge-to-crossing incidence lists. Store crossings in a heap keyed by $(-s_i, -\ell_i, i)$, using the opposite sign for a minimum heap. When a crossing is processed, increment the shared count of each still-active neighboring crossing and push its new key. Discard inactive or stale heap entries on pop. Each diagram edge causes at most one such increment. Thus there are $O(n)$ heap operations, each costing $O(\log n)$.

Proposition 7.1. *For the same starting crossing, the new planner returns exactly the original order in $O(n \log n)$ time instead of $O(n^2)$. Trying r starts gives $O(rn \log n)$ instead of $O(rn^2)$.*

Proof. The score reduction preserves the total order, including index tie-breaks. The heap contains a current key for every active crossing; stale keys cannot cause an incorrect choice because they are rejected. Incidence updates give exactly the number of processed neighbors, with multiplicities. Induction on the chosen prefix proves equality of the orders. The operation count gives the complexity bound. $\square$

The default start-selection rule and width-profile comparison are unchanged. The old stride convention may produce slightly more than the requested number of starts, but still $O(r)$. Requesting all starts is $O(n^2 \log n)$ in the new version, not $O(n \log n)$. The benchmarks use the bounded-start policy of the recognizer. This is not an optimal-width algorithm or a separator algorithm.

8 Compiling the characteristic-two cobordism algebra

8.1 Representation and cancellation

The scan is Bar–Natan’s local simplification method [4]. Objects are frontier matchings, with homological degree and multiplicity. Closed circles are delooped into two summands corresponding to $1, x$ in $\mathbb{F}_2[x]/(x^2)$. A morphism is an $\mathbb{F}_2$ sum of dot-subsets on the boundary circles of a cobordism; addition is symmetric difference of these finite sets. Invertible differential entries between equal matchings are cancelled after each crossing.

If $\phi : B \rightarrow C$ is an invertible differential block, with neighboring blocks $\delta : A \rightarrow C$ and $\gamma : B \rightarrow F$, Gaussian cancellation replaces the $A \rightarrow F$ block by

$$d_{AF} + \gamma\phi^{-1}\delta. \quad (11)$$

The usual minus sign equals plus in characteristic two. The eliminated pair is contractible, so this preserves homotopy type. We retain the old LIFO pivot policy; improvements in the measurements are not attributed to an unimplemented minimum-fill pivot heuristic.

8.2 Every unit is its own inverse

If a matching has r arcs, its endomorphism ring has the form

$$R_r = \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2). \quad (12)$$

Its units are exactly $1 + \nu$ with zero constant term in ν .

Lemma 8.1. *Every such unit satisfies $(1 + \nu)^{-1} = 1 + \nu$.*

Proof. In a commutative characteristic-two ring the square of a sum is the sum of its squares. Every nonconstant square-free monomial has zero square, because it contains some x_i and $x_i^2 = 0$. Hence $\nu^2 = 0$ and $(1 + \nu)^2 = 1$. $\square$

The new inverse therefore validates the constant term and copies the set of terms. Its cost is linear in the stored representation, rather than requiring morphism multiplications. Importantly, the old geometric-series loop already terminates after at most two products in this ring: we do not claim that it required r or exponentially many iterations. The improvement is eliminating these expensive products altogether.

For endomorphism multiplication, square-free monomials indexed by dot sets S, T satisfy

$$x_S x_T = 0 \quad (S \cap T \neq \emptyset), \quad x_S x_T = x_{S \cup T} \quad (S \cap T = \emptyset).$$

The optimized path uses this rule directly and XORs equal output monomials. Identity multiplication is also short-circuited.

8.3 Separating topology from dot data

For a general composite $a \rightarrow b \rightarrow c$, the topology depends on the three matchings, not on the coefficient terms. The new `_composition_plan` compiles once the connected components of the glued surface, their Euler characteristics, their outgoing boundary-circle sets, and maps from each incoming dot position to a component bit.

For each pair of input terms, the implementation only has to form the mask of dotted components. A repeated bit means two dots lie on the same component and the contribution is zero. A component of genus at least one is also zero: its handle map is multiplication by $2x = 0$ over $\mathbb{F}_2$. For a planar component with $q > 0$ outgoing circles, zero dots gives the sum of the q monomials in which exactly one outgoing circle is undotted; one dot gives the all-dotted monomial. A closed planar component evaluates to one precisely when it has one dot. These are exactly the multiplication, comultiplication, and counit rules of the same Frobenius algebra. The optimized expansion changes their organization, not their values.

New morphism and crossing-entry caches store frozen values and return copies where the public internal API expects mutable sets. This prevents a caller's subsequent XOR or clearing operation from poisoning a cached morphism. All new caches have explicit LRU entry ceilings. The existing circles/glue caches also receive finite entry ceilings, and `clear_caches()` resets them. These limits bound the number of entries, *not* the byte size of a single large entry. No fixed memory bound in megabytes is claimed.

The limits are 32768 for circles and glue, 8192 for composition plans, 32768 for mask expansions, 16384 for completed compositions, and 8192 for crossing entries. They are engineering choices, not complexity constants required by correctness. Eviction merely causes recomputation.

8.4 Why the final rank detects the unknot

The backend computes the total unreduced rank over $\mathbb{F}_2$ and divides by two. Here is the algebraic justification for that division. On each resolved state let X multiply the basepoint-circle factor by x , and let ν be the sum of the operators ∂_x on all circle factors, where $\partial_x(1) = 0$ and $\partial_x(x) = 1$. Over $\mathbb{F}_2$ both are chain maps: this follows directly by checking multiplication and comultiplication on the basis $1, x$. They satisfy

$$X^2 = \nu^2 = 0, \quad X\nu + \nu X = 1.$$

The reduced subcomplex is $\text{im } X$. Every chain splits uniquely as $X\nu c + \nu Xc$, giving

$$C \cong \text{im } X \oplus \nu(\text{im } X),$$

with the two summands isomorphic by X and ν after suppressing the quantum shift. Thus total unreduced rank is twice total reduced rank; see also the characteristic-two structure discussed in [7].

Kronheimer–Mrowka prove that reduced rational Khovanov rank one detects the unknot [6]. The integral reduced complex is free at chain level, so the universal coefficient theorem gives

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) \geq \dim_{\mathbb{Q}} \widetilde{\text{Kh}}(K; \mathbb{Q}).$$

The rational rank is at least one: the reduced graded Euler characteristic specialized at one is one for a knot. Therefore reduced $\mathbb{F}_2$ rank one forces rational rank one and hence triviality. Conversely, the unknot has reduced rank one over $\mathbb{F}_2$. This proves the exact terminal criterion used here.

9 Why frontier width alone cannot bound the explicit backend

The original README correctly warns in one place that matching multiplicities need control, but elsewhere describes the exponential part as depending on width rather than on crossing number. For the explicit minimal-complex representation, a uniform $\text{poly}(n)f(w)$ bound is false.

Let C be the supplied Conway diagram and let C_k be the connected sum of k copies, constructed by `tools/families.py`. It has $11k$ crossings. The independent reduced cube and both scanning backends compute

$$\dim \widetilde{\text{Kh}}(C; \mathbb{F}_2) = 33.$$

For connected sums the reduced complexes tensor over the ground field, with the marked-circle factors identified. On state bases this identifies the unmarked factors of the two diagrams; crossing differentials act on their own factor, giving the tensor differential. The Künneth theorem over a field consequently gives

$$\dim \widetilde{\text{Kh}}(C_k; \mathbb{F}_2) = 33^k. \quad (13)$$

This is the standard reduced connected-sum construction [8]. For $k = 2$, the delivered tests and baseline benchmark independently of the filter path verify reduced rank 1089 and unreduced rank 2178.

The factors can be scanned consecutively with bounded frontier size: between fixed-size factors only the connecting strands are exposed, and within a factor only a constant number of additional edges occurs. Nevertheless the fully reduced closed complex has $2 \cdot 33^k$ explicit objects. At that final stage there is one possible matching—the empty matching—but an exponential multiplicity. Counting possible matchings cannot bound those multiplicities. This is an output-size lower bound for this representation, not a lower bound for every possible unknot recognizer or a compressed rank algorithm.

There is a corresponding positive result for the new filter. The exact normalized bracket evaluation at $A = 2$ of the supplied Conway diagram is

$$r = \frac{\mathcal{B}_C(2)}{\delta(-2^3)^{w(C)}} = \frac{970142522911}{65536} > 1. \quad (14)$$

The normalized bracket is multiplicative under connected sum: joining the marked state circles reduces their combined circle count by one, cancelling one factor of δ from the product, and writhe is additive. Hence the corresponding value for C_k is $r^k \neq 1$.

Corollary 9.1 (A genuine family-specific separation). *With a factor-consecutive bounded-width order and sufficient transition/state budgets, the implemented exact integer bracket filter rejects every C_k in polynomial bit time. An algorithm that explicitly materializes the closed minimal Khovanov complex for the same family requires exponentially many objects.*

Proof. Equation (14) and multiplicativity give a mismatch for every $k \geq 1$. Bounded width gives a constant matching-state bound; coefficients have $O(k)$ bits. There are $O(k)$ crossing steps, hence polynomial bit work. Equation (13) proves the explicit-object lower bound. $\square$

This corollary does not assert that the default greedy order and default caps have the same bound for every k . It describes the exposed low-level filter with the indicated order and budgets. The family tests check exact multiplicativity for $k = 1, \dots, 5$. A fixed modular evaluation alone

would not justify the corollary: in a finite field nonzero values have finite multiplicative order, so powers can collide with one. That is a concrete reason for including exact integer mode.

The global worst-case upper bound of the complete recognizer remains $2^{O(n)}$, with conservative polynomial factors absorbed into the exponent. The ordinary resolution cube has exponentially many states and enhanced-state basis elements; intermediate delooping and finite-field elimination can be bounded by exponential-size matrices and exponentially many dot terms. The polynomially capped new filters do not alter that class. A general quasi-polynomial guarantee would require an additional structural algorithm, not just these optimizations.

10 Paired measurements, including regressions

10.1 Protocol

`benchmarks/run.py` generated 23 cases and ran each engine three times, for 138 separate timed calls. The original ten fixtures are included, together with connected-sum stress cases and raw-backend/planning diagnostics. Every call uses a fresh subprocess and cold caches. Engine order alternates across cases and repetitions. The child interpreter uses `-S` to avoid loading unrelated site initialization; both packages need only the standard library. `PYTHONHASHSEED=0` is fixed. The timer is `time.perf_counter` around the requested function, excluding interpreter/import startup and the first `Diagram.from_json` parse. Validation performed again inside the new recognizer or raw backend is included in its time.

The environment reports Python 3.13.5, GCC 14.2.0, Linux x86-64 with glibc 2.41, and an AMD EPYC 9V74 80-Core Processor; five logical CPUs are visible to the environment. The computation is serial. These are shared-container wall clock measurements, not controlled hardware cycle counts or statistical confidence intervals. Three trials provide a reproducible sample, not a claim of universal speedup.

The cooperative budget is three seconds and the parent's hard subprocess limit is 5.5 seconds. The original raw backend starts its internal deadline *after* order selection; this is why a completed original 1024-crossing call can take more than three seconds in total. The new backend starts before planning. These soft timers are not exact process deadlines. All 138 calls returned; 126 completed, and 12 returned resource-limit results. There were no hard timeouts or execution errors in this run.

The JSON contains every input, option, result, trial time, outcome, and reported peak resident-set value. RSS units are platform-specific and it includes the process environment, so it is not used as a memory-speedup claim. The CSV includes minima, maxima, and medians. Table source is generated from that JSON by `benchmarks/make_tables.py`, rather than transcribed by hand.

Case	n	Original (ms)	New (ms)	Ratio
Crossing-free unknot	0	0.020	0.026	0.75
Trefoil	3	0.092	0.123	0.75
Figure-eight	4	0.134	0.128	1.05
Hard unknot (8 crossings)	8	1.629	2.242	0.73
Kinoshita–Terasaka	11	39.313	1.018	38.61
Conway	11	39.304	0.822	47.79
$T(3, 5)$	10	0.591	0.912	0.65
Unknot braid (40 crossings)	40	6.681	7.999	0.84
Determinant-one grid knot	10	0.721	0.957	0.75
Scrambled grid unknot	6	0.259	0.288	0.90
Conway # Conway	22	1399.518	2.462	568.54
Three Conway summands	33	LIMIT	3.379	–
Four Conway summands	44	LIMIT	4.112	–

Table 1: Full recognition, defaults in both versions. Ratio is original/new; values below one are regressions. Each time is the median of three completed cold-process calls. Limit means all three calls returned UNKNOWN.

Case	n	Original (ms)	New (ms)	Ratio
Conway	11	35.973	13.269	2.71
Kinoshita–Terasaka	11	39.206	15.090	2.60
Hard unknot (8 crossings)	8	2.476	2.165	1.14
$T(3, 5)$	10	10.206	9.958	1.02
Unknot chain (256 crossings)	256	231.987	20.751	11.18
Unknot chain (1024 crossings)	1024	3601.461	84.039	42.85
Random 5-braid (36 crossings)	36	LIMIT	LIMIT	–

Table 2: Raw Khovanov backend, without recognition preprocessing or invariant filters. Both engines use the same deterministic scan-order policy.

Case	n	Original (ms)	New (ms)	Ratio
Order only: 256-crossing chain	256	217.852	5.844	37.28
Order only: 1024-crossing chain	1024	3550.885	27.065	131.20
No R1/R2/descending: hard unknot	8	2.907	3.203	0.91

Table 3: Planning and forced-fallback diagnostics. In the last row only Reidemeister reduction and descending recognition are disabled; invariant filters are still enabled.

10.2 What the results establish

The strongest completed end-to-end result is the two-Conway connected sum: 1.399518 seconds becomes 0.002462 seconds, a median ratio of 568.54. This compares recognition with recognition, not an unfiltered old homology calculation against a new shortcut with different requested output. Both versions return **KNOTTED**. The baseline reaches reduced Khovanov rank 1089; the new version proves a bracket mismatch before constructing that complex.

Conway and Kinoshita–Terasaka each have trivial Alexander polynomial in these fixtures. The full recognizer is respectively 47.79 and 38.61 times faster. The raw backend still speeds up by 2.71 and 2.60 on the same PD inputs, showing that the implementation is not merely a wrapper around an unchanged exponential kernel.

For the chain braid $\sigma_1\sigma_2\cdots\sigma_n$ on $n+1$ strands, raw backend time changes from 0.231987 to 0.020751 seconds at $n=256$, and from 3.601461 to 0.084039 seconds at $n=1024$. Order-only times isolate the asymptotic planner change. These raw cases intentionally bypass Reidemeister recognition: the chain is easy to simplify, so they are backend scaling tests, not claims that an otherwise difficult unknot has been solved.

The results are not uniformly positive. On the hard eight-crossing unknot the default full pipeline is about 1.38 times slower even though raw backend computation improves modestly. The extra invariant stages do not reject an unknot and therefore add overhead. The small $T(3,5)$ and grid cases are also slower because the original polynomial stage was already cheap. In absolute terms these changes are small, but they are not omitted from the tables.

Three- and four-Conway sums finish quickly in the new recognizer, whereas all baseline trials exhaust the selected resource budget. No completed-baseline speedup is reported for these censored cases. On the selected 36-crossing random five-braid, *both raw backends exhaust the budget*. This remains an unresolved performance case for the present code. Earlier timing claims in the supplied archive are not substituted for fresh measurements.

10.3 Counters that should not be compared naively

The optimized `compositions` counter counts an explicit composition once where it is actually called. The old counter increments by two inside each inner elimination pair even though the first half-composition is outside that inner loop. Their numeric values have different meanings; we do not use their ratio as an operation-count speedup. Object counts, final ranks, and scan orders retain their mathematical meanings. Bounded caches can also change warm-process behavior; all headline data above are cold-process data.

11 Validation and the limits of that validation

The final command

```
python -S -m unittest discover -s tests -v
```

passes **42 tests in 9.444 seconds** in the recorded run. This count is unittest test methods, not 42 individual diagrams: many methods contain exhaustive or randomized families. The source and full log are included.

- The original 19 tests are retained with narrowly scoped changes to expected method names and to filter disabling in backend-resource tests. The untouched original test file is preserved in [reference/](#).
- On 168 one-component three-braid words of lengths two and four, and additional deterministic random samples, new verdicts are compared with an independent reduced resolution cube. A separate 80-diagram test compares raw rank and rank-by-cube-degree with the original backend, with $d^2 = 0$ checks.
- The bracket is compared to an independent global state sum over exact `Fraction` values on those 168 words, 120 random diagrams, and the crossing-free case. Transition gluing is also compared to the separate cobordism transition routine. Mirrors, arbitrary scan orders, R3 braid relations, and positive/negative Markov stabilizations are tested.
- All 2168 pairs of basis morphisms for triples of the five noncrossing matchings on six endpoints are composed by both kernels. Another 250 random eight-endpoint combinations test agreement and associativity. Every unit of endomorphism rings through three arcs is checked, including its old inverse.
- Every starting crossing is checked for planner equivalence on 100 random diagrams. Descending results match the old implementation on the exhaustive family and 150 random diagrams. Writhe and Alexander behavior are checked under random two-slot row rotations and row permutations.
- Sparse determinants are compared with permutation-expansion determinants for sizes zero through five. Modular Alexander minors are checked against full exact polynomials on the exhaustive family and 200 random diagrams. Deliberate modular collisions, budget exhaustion, malformed orders, invalid diagrams, cache mutation, witness tampering, and command-line exit codes are covered.
- The independent reduced cube computes rank 33 for both Conway and Kinoshita–Terasaka. Connected-sum tests verify the 22-crossing fixture, rank 1089, and exact integer bracket powers for the first five Conway sums.

Random tests use fixed seeds and include diagrams that may represent the same knot; no claim about an exhaustive knot census is made. The independent cube and bracket state sum avoid the optimized composition and gluing code, which reduces the risk of a shared implementation error. The PD parser and ordinary Python arithmetic remain shared trusted infrastructure. This is not a Lean formalization, a proof of correctness of the Python interpreter, or exhaustive verification for all inputs. The mathematical arguments explain the intended algorithm; the tests substantiate the implementation on the stated domains.

12 Using the package and interpreting evidence

12.1 Commands

Run from the extracted package root; installation is optional.

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/conway_sum_2.json --output result.json
python -m fastunknot verify examples/conway_sum_2.json result.json

# Compute homology itself, not just a knottedness obstruction.
python -m fastunknot khovanov examples/conway.json --check-d2

# Force the complete backend; no preprocessing or invariant shortcut.
python -m fastunknot recognize examples/hard_unknot_8.json \
    --no-reduction --no-descending --no-alexander --no-jones --check-d2

# An exact one-point obstruction, not a complete recognizer.
python -m fastunknot bracket examples/conway.json --integer

# Reproduce and regenerate the recorded tables.
python benchmarks/run.py --repeats 3 --seconds 3
python benchmarks/make_tables.py
cd docs
pdflatex -interaction=nonstopmode -halt-on-error unknot_acceleration.tex
pdflatex -interaction=nonstopmode -halt-on-error unknot_acceleration.tex
```

The Python API retains `Diagram`, `recognize`, and `khovanov_rank`. A supplied raw scan order must be a permutation of all crossing indices; duplicate, missing, Boolean, and out-of-range indices are rejected. Directly constructed `Diagram` objects are revalidated by the recognizer. The default result JSON explicitly sets `quasipolynomial_guarantee` to false.

12.2 Limits and ablation switches

`-no-alexander` disables both the modular determinant and full Alexander polynomial; it does not disable the independent bracket tests. `-no-modular` disables just the modular determinant, not the bracket. `-no-jones` disables both bracket evaluations. `-no-integer-jones` disables only the exact integer follow-up. Setting the transition budget to zero forces the bracket stage to pass through without a decision. The low-level Python bracket function accepts a custom order and `modulus=None` for exact integer mode; omitting its caps gives an uncapped filter computation.

`-max-objects` caps the Khovanov complex, not the invariant filters. Thus a knottedness obstruction may succeed even with `-max-objects 1`. `-seconds` is cooperative. Checks occur around stages and within elimination loops, but a large single operation, validation, planning, or Reidemeister simplification can overshoot it. A strict wall-clock or memory policy requires an operating-system process limit, as in the benchmark runner. Limits should never be interpreted as a complexity guarantee.

Recognition exit status is zero for either exact verdict, two for invalid input, and three for UNKNOWN. Verification returns zero for accepted evidence and four for rejected evidence. Successful `bracket` execution returns zero even when `detected` is false; inspect its JSON fields rather than interpreting this utility's exit status as a knot verdict.

12.3 What the verifier certifies

A modular or integer **KNOTTED** result contains its evaluation parameters, computed value, required unknot value, scan order where relevant, and the R1/R2 reduction trace. **verify_modular_result** first replays legal moves on the supplied original input, then recomputes the obstruction on the reduced diagram. Its historical function name includes “modular”, but it also supports exact integer bracket evidence.

For the Conway example at $A = 2$ and $p = 2147483647$, the recorded writhe is -1 , the bracket residue is 142097288, and the required unknot residue is 1140850688. They differ. The corresponding result and verification output are in **results/witnesses/**. This is evidence about the original input, not merely an unrelated simplified diagram.

The verifier does not accept arbitrary Khovanov-rank JSON as a proof, and it is not a small-certificate or polynomial-time verifier claim: bracket verification recomputes the scan and can be expensive. The record is a reproducible exact obstruction, not a cryptographic attestation or an adversarially hardened network protocol.

13 Conclusions and remaining engineering opportunities

The supplied implementation meets the practical-performance alternative in the request, with a large completed same-task speedup, a faster backend, proved asymptotic improvements in ordering and descending detection, and a family-specific separation between exact bracket rejection and explicit Khovanov materialization. It also exposes a concrete mistake in treating frontier matching count as a bound on complex size.

The largest outstanding performance issue is inputs on which all inexpensive obstructions are inconclusive and the minimal complex has large multiplicities. The present code still constructs that complex. Potential next engineering steps include factorization across proven connected-sum cuts, compressed representations of repeated summands, incremental simplification, and different elimination pivot policies. None is silently credited to this implementation or to the measurements. A global quasi-polynomial result would need a proved structural replacement, such as a fully constructive accelerated hierarchy with controlled depth and coordinate cost. The code, proofs, measurements, and limitations in this delivery are kept separate so that such future progress can be assessed against a reproducible baseline.

References

- [1] *Knots.zip*, user-supplied archive, inspected 18 September 2026. Baseline: `fast/fastunknot/`; original tests: `fast/tests/test_fastunknot.py`. Literature materials: `docs/quasipolynomial-talk.pdf` and `docs/arXiv-2607.23350v1/algorithm-incompressible-250726.tex`.
- [2] M. Lackenby, *Unknot recognition in quasi-polynomial time*, talk slides, February 2021, 109 physical PDF pages. Official author copy: <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [3] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. In particular Section 9, pp. 34–35, and the refinements in Sections 10–14. <https://arxiv.org/abs/2607.23350>.
- [4] D. Bar–Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications **16** (2007), 243–255. <https://arxiv.org/abs/math/0606318>.
- [5] L. H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987), 395–407. DOI: [10.1016/0040-9383\(87\)90009-7](https://doi.org/10.1016/0040-9383(87)90009-7).
- [6] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. <https://arxiv.org/abs/1005.4346>.
- [7] A. Shumakovitch, *Torsion of the Khovanov homology*, Fundamenta Mathematicae **225** (2014), 343–364; arXiv revision dated 19 June 2018. <https://arxiv.org/abs/math/0405474>.
- [8] M. Khovanov, *Patterns in knot cohomology I*, Experimental Mathematics **12** (2003), 365–374. <https://arxiv.org/abs/math/0201306>.